AETHERIA

Century-scale decentralized MMO architecture

Security review and revised reference specification | Revision 2

Reviewed 8 October 2026. Source: the supplied 16-page architecture, including its implementation directive and bibliography. Original page numbers in the findings refer to that source version.

Verdict: the original design is not ready for production. It contains mechanisms that are useful in isolation, but combines them into security guarantees they do not establish. This revision replaces those guarantees with conditional requirements and a conservative implementation profile.

The highest-risk gaps are unauthenticated consensus membership, conflicting offline histories, unsafe cross-shard ownership, finality without data availability, selective randomness disclosure, and recovery claims unsupported by storage placement. Permanent kernel freezing would also prevent repair of a discovered consensus bug.

What this revision changes

- Separates open player participation from validator admission and spatial discovery from consensus authority.
- Makes canonical progress stop when required quorums or data are missing. Offline play is a disposable local branch by default.
- Defines a single ownership authority, durable transaction decisions, bounded deterministic execution, replay protection, and authenticated recovery.
- Removes claims that transcoding eliminates all covert channels, parity fragments eliminate liability, or a seed and fountain code guarantee century-scale survival.
- Adds an adversarial test matrix, implementation sequence, and explicit release blockers.

Assurance boundary. This is an architecture-level review and proposed redesign. No source code, executable protocol, deployment, packet trace, benchmark, formal model, or proving circuit was supplied. No penetration testing or implementation verification was performed. A requirement written here is not evidence that it is implemented or proven. No document review can certify that every exploit has been eliminated.

Preserved goals. Deterministic physics; peer-operated infrastructure; procedural terrain; player and agent action parity; portable world state; and long-term independent operation. Continuous availability, fully permissionless voting, permanent immutability, and absence of governance are not simultaneously assumed.

Reading guide and change status

Pages 3-7: findings register. Thirty design findings link the original claims to attack paths, required controls, and remaining limitations. These are design risks, not demonstrated vulnerabilities in a running product.

Pages 8-23: revised architecture. Threat model, deployment profile, consensus, deterministic execution, input ordering, transfers, spatial seams, isolation, randomness, transport, sandboxing, content, storage, recovery, economy, and key lifecycle.

Pages 24-25: verification and release gates. Counterexamples and failure injection scenarios to test before claiming security.

Page 26: replacement engineering directive. An implementation brief that supersedes the source document's request for immediately production-ready primitives.

Pages 27-28: evidence and references. Primary sources used to check foundational claims, with limits on what each supports.

How to interpret requirements

MUST and MUST NOT identify proposed mandatory protocol behavior. SHOULD marks a recommendation with a documented exception. These words describe the target design; they do not confer assurance on an unbuilt system.

Severity is qualitative. **Critical** means possible canonical history corruption, duplicated ownership, or irrecoverable loss. **High** means substantial manipulation, denial of service, or boundary compromise. **Medium** means a material limitation or misleading guarantee. Severity depends on deployment; no CVSS score is assigned without an implementation.

Design decisions made in this revision

- The launch profile uses a disclosed federation of independent operators. Players remain open; validator membership is explicitly admitted. This is a deliberate departure from fully ownerless operation, because gameplay age alone does not establish independent identities.
- A certified control journal records membership, shard authority, recovery checkpoints, and cross-shard decisions. It is internal to this network and does not require a public blockchain. It is still an operational infrastructure dependency.
- Canonical offline progression and arbitrary executable player code are disabled at launch. Both require separate specifications and verification before activation.
- Security updates are permitted through visible protocol epochs. A permanently frozen implementation cannot promise safe repair.

Status: revised, not certified. All findings remain unverified until the applicable implementation tests and independent review pass. This document provides mitigations, not a security sign-off.

Findings 01-06 | Canonical state and randomness

F01 | Critical | Gameplay stake does not establish Sybil resistance

Source: pp. 5-7, 13. An attacker can age many automated identities, buy accounts, or concentrate local avatars. Historical activity and cheap in-game progress do not prove distinct operators. DHT-selected auditors can be captured through the same identity or routing mechanism. **Revision:** explicit operator admission and certified voting keys at launch; gameplay progress confers no vote. Permissionless admission remains a separate research problem. Deterrence is not a security proof. [S1]

F02 | Critical | A supermajority signature is not a consensus protocol

Source: pp. 4, 13. Signing a delta root omits view changes, lock rules, durable votes, data availability, and membership changes. Counting only online or nearby nodes lets an attacker shrink the electorate and obtain conflicting certificates. **Revision:** a reviewed BFT protocol with fixed epoch membership, durable signing state, exact commit rules, and certified reconfiguration. Never lower the threshold during an outage. [S2, S3]

F03 | Critical | Two valid offline proofs can conflict

Source: pp. 4, 13. A player forks from one accepted inventory and proves two individually legal histories, then submits them to different chunks. A proof of execution does not establish exclusive ownership, completeness of inputs, or unique canonical history. **Revision:** no canonical offline progression at launch. Any later extension requires finalized exclusive leases, consumed-once exit records, and authoritative reconciliation. [S4; review inference]

F04 | Critical | Spatial partitions can double-spend shared entities

Source: pp. 2-4, 12-13. Adjacent chunks or disconnected quorums may each treat the same avatar, inventory, or projectile as writable. Overlapping membership alone does not determine a unique owner. **Revision:** every entity has one certified authority and ownership version. Transactions lock versioned objects and use a durable common decision. A minority partition cannot commit shared state.

F05 | High | Commit-reveal permits selective abort

Source: pp. 4-5, 13. The last revealer computes the combined result after seeing the other's nonce, then withholds an unfavorable reveal. No pre-image attack is necessary. A timeout fallback may itself become the alternative outcome the attacker chooses. **Revision:** remove player-controlled reveal from consequential outcomes; lock actions before a designated future threshold-beacon result. Missing beacon means pending resolution, not a substitute seed.

F06 | High | Valid computation does not prevent outcome grinding

Source: pp. 4-5, 13. A client knowing a public seed can try many legal offline input sequences and submit only the best. VDF evaluation does not create entropy and does not remove input choice or faster-hardware advantage. **Revision:** commit eligible actions before entropy release, enforce one authorized attempt per resource/version, and disallow offline contested or valuable random outcomes. [S5; review inference]

Findings 07-12 | Availability, storage, and boundaries

F07 | Critical | A certified root can refer to unavailable data

Source: pp. 4, 6, 8. Validators can attest to a hash while withholding inputs, deltas, or reconstructible state. Sentinels cannot replay unavailable data; joining peers cannot recover from a root alone. **Revision:** validators persist complete bounded transition data before voting; publish availability commitments and independently test retrieval. Audit witnesses are additional detectors, not a substitute for honest validators or retained data.

F08 | Critical | Arbitrary 99.9% loss is not an erasure-code guarantee

Source: pp. 7-9, 13. The remaining peers may hold no symbols for a required block, or only duplicates. RaptorQ reconstructs from sufficient valid symbols, not from an arbitrary fraction of surviving node identities. **Revision:** specify per-block placement, independent failure domains, rank/decoding sufficiency, integrity, and recovery tests. Distinguish random survival estimates from adversarial guarantees. [S6]

F09 | Critical | Permanently frozen code preserves exploitable defects

Source: pp. 1-2, 11-12. An exploitable arithmetic, consensus, or verifier bug cannot be fixed under an absolute ban on semantic updates. A kernel hash proves identity, not correctness. **Revision:** stable documented semantics with visible, certified security epochs, compatibility tests, archived versions, and an explicit fork policy. Do not label the current kernel formally verified without proof artifacts.

F10 | High | Discovery and committee selection are grindable

Source: pp. 2, 6. Attackers generate DHT IDs near chunk or auditor keys, advertise false addresses, and eclipse a joining peer. Selecting the longest-lived or fastest relay favors well-resourced colluders. **Revision:** discovery is untrusted routing only. Authenticate records, diversify bootstrap paths, pin checkpoints, and select authority solely from the certified registry. Relay status never confers a vote.

F11 | High | A fixed 16-meter halo misses interactions

Source: pp. 2-3, 12. Fast projectiles, explosions, teleportation, and prolonged rollback can cross the halo without simultaneous validation. Duplicate simulation can apply damage twice. **Revision:** swept collision coverage derived from speed, acceleration, interaction radius, and rollback horizon; one event authority and event ID; remote effects require certified handling. Unavailable boundaries block irreversible effects.

F12 | High | Two-phase commit can strand locks

Source: pp. 3, 12. A failed or dishonest coordinator can leave resources permanently prepared; independent local timeouts can unlock an object after the other side commits. **Revision:** durable BFT-certified decision journal, logical expiry, idempotent application, and no unilateral timeout unlock. Consensus-backed decisions trade blocking safety for availability. [S7]

Findings 13-18 | Execution and message integrity

F13 | High | Fixed-point arithmetic alone is insufficient

Source: pp. 1-2, 4-5, 12. Entity iteration, map ordering, ties in collision resolution, terrain generation, host calls, thread scheduling, and trap handling can still diverge. **Revision:** a normative deterministic execution profile, canonical input ordering, fixed instruction/resource accounting, and cross-architecture golden state roots. Rendering and planners stay outside consensus.

F14 | High | Numerical edge cases can mint or corrupt state

Source: pp. 5, 12, 14. Q64.64 products need wide intermediates; wraparound, division by zero, inconsistent negative rounding, and zero-length vectors can change ownership or conservation outcomes. **Revision:** signed i128 representation, exact i256 intermediates or reviewed equivalent, specified rounding, bounded ranges, and atomic rejection on arithmetic faults. Ban silent wrapping and undocumented saturation.

F15 | High | Wasm and proofs expose computational DoS

Source: pp. 1, 4, 12-14. Infinite loops, memory growth, pathological meshes, or large malformed proofs can exhaust a validator. Sandboxing does not itself meter total computation or eliminate runtime bugs. **Revision:** fuel, memory, table, stack, witness, proof, and queue limits; isolated workers; cheap validation before expensive operations. Disable executable user modules at launch. [S8]

F16 | High | Typed schemas do not make parsing safe

Source: pp. 6, 13. Oversized lengths, malicious offsets, recursive objects, duplicate fields, malformed encodings, and decompression bombs remain possible in typed formats. Rejection necessarily occurs after some bytes are received and inspected. **Revision:** framing limits before allocation, canonical bytes, recursion/traversal limits, deterministic rejection, and differential fuzzing of all trust-boundary decoders.

F17 | High | Signed inputs can still be replayed

Source: pp. 4, 9, 14. A valid signature lacking world, epoch, session, sequence, and object-version binding can be reused after reconnect, migration, or restart. Repeated trade or harvest intents can be applied twice. **Revision:** domain-separated envelopes, deterministic sequence handling, consumed-once transaction IDs, persistent deduplication, and explicit session/key rotation.

F18 | High | Proof acceptance can verify the wrong claim

Source: pp. 4, 13. A proof may bind the wrong program, old state, attacker-selected public outputs, or a test verifier. Wasm execution and a RISC-V zkVM are not interchangeable without a specified bridge. **Revision:** pin program/verifier IDs; bind every public input; reject fake/dev receipts and unresolved assumptions; enforce bounded output validation. Prove interpreter equivalence before bridging VMs. [S4, S9]

Findings 19-24 | Content, capacity, and information leakage

F19 | High | Content limits cannot eliminate arbitrary data channels

Source: pp. 6-7, 13. Allowed palette choices, geometry, item layouts, action timing, keys, and signatures can encode information. Normalizing a subset of bits does not eliminate the remaining choices. **Revision:** bound carrier capacity with vetted primitives, strict metadata restrictions, low upload quotas, and finite retention. Remove the claim of technical impossibility. Content moderation remains partly operational.

F20 | Medium | Encryption and parity do not establish legal immunity

Source: pp. 6, 11. Fragments remain collectively reconstructible; public spatial keys provide little confidentiality. Transient-key loss can also destroy recoverability. No technical encoding establishes a universal legal classification. **Revision:** remove the liability guarantee; minimize replicated user content, allow operator storage policies, and obtain jurisdiction-specific advice before deployment. This review makes no legal determination.

F21 | High | Asset processing creates a second execution boundary

Source: pp. 6, 13. Tiny malformed meshes can still trigger expensive triangulation, renderer bugs, extreme coordinates, or degenerate collision shapes. Player-selected paths or external references create additional access risks. **Revision:** built-in assets at launch; bounded sandboxed optional transcoding; no URLs, scripts, filenames, external shaders, or user-selected collision code. Client rendering is separately hardened.

F22 | High | Sparse deltas and AI upkeep do not bound real storage

Source: pp. 8-10, 13-14. Attackers produce endless valid edits, entities, and logs. In-game food or decay does not pay for disk, network traffic, or proof verification; an append-only world grows indefinitely. **Revision:** real byte/CPU/egress admission budgets, entity and state-growth caps, checkpoints, finite mandatory logs, repair headroom, and an explicit archival retention contract.

F23 | High | Crowds and shard churn can overwhelm validators

Source: pp. 2-3, 12. Repeated split/merge triggers, boundary oscillation, collocated players, or high fanout can defeat locality. Smaller chunks do not guarantee fewer interactions. **Revision:** split hysteresis, minimum shard lifetime, certified topology transitions, per-region work budgets, and bounded interaction fanout. Relay compression is a performance hypothesis to benchmark, not a security guarantee.

F24 | High | Public deterministic state exposes hidden information

Source: pp. 2, 4, 8. A malicious client inspecting authoritative state or the genesis seed can locate hidden enemies, predict procedural resource veins, and automate search. AOI subscription filters are not secrecy when peers possess the data. **Revision:** accept public-state visibility at launch and design mechanics accordingly. Genuine fog of war requires a separate trusted or cryptographic confidentiality design; zk execution alone does not supply it.

Findings 25-30 | Operators, agents, and long-term integrity

F25 | High | Bots and agent keys have unspecified authority

Source: pp. 9-10, 14. Signatures prove control of keys, not humanness, fair play, or that an authorized planner chose an action. Agent host loss, duplicate hosts, compromised planners, or prompt injection can misuse inventory and identity. **Revision:** bots are allowed under explicit quotas; agents receive revocable action capabilities and spending limits, never validator or update keys. Host transfer is versioned and atomic. Sovereign agents may cease when their host disappears.

F26 | High | Economic conservation and time are underspecified

Source: pp. 9-10, 14. Negative quantities, recycling loops, integer rounding, infinite procedural space, duplicate harvests, and forged elapsed time can create unlimited wealth despite conserved mass. Host clocks can accelerate decay or rewards. **Revision:** bounded active world/resource supply; integer ledgers and resource versions; exact recipe accounting; certified logical ticks; capped catch-up work. Conservation does not guarantee economic balance.

F27 | High | Key theft, replayed votes, and unsafe penalties

Source: pp. 6, 9, 14. Stolen or lost Ed25519 keys can impersonate owners. Validators that restart without durable locks can double-sign. An accusation or unavailable audit evidence can be used to slash honest players. **Revision:** distinct root/session/validator keys, hardware-backed validator custody where possible, write-ahead signing state, and verifiable evidence plus certified penalty decisions. Key recovery is an explicit trust policy. [S10]

F28 | Critical | Old keys can manufacture plausible ancient history

Source: pp. 1, 6, 8, 11. A long-offline client without a trusted recent checkpoint may accept a counterfeit chain signed by compromised retired validators. More signatures or the highest claimed height do not identify the intended current world. **Revision:** checkpoint pinning, authenticated membership lineage, signer key evolution/retirement, and independent archive corroboration. If current lineage is unavailable, recovery is an explicit new fork, not silent finality.

F29 | High | Century-scale security needs maintenance and recovery

Source: pp. 1, 11-14. Cryptographic aging, dependency compromise, malicious updates, runtime vulnerabilities, and loss of bootstrap or NAT relays can end operation. No plan funds or replaces real infrastructure. **Revision:** documented stewardship and replacement paths, reproducible builds, dependency audits, diverse bootstrap/relay operators, archived tooling, versioned cryptography, and migration before algorithms fail.

F30 | High | Censorship and ordering remain exploitable

Source: pp. 3-4, 6. A relay or proposer can omit, delay, reorder, or leak inputs while producing valid state. Late-input rollback can change a past combat outcome; auditors detect illegal execution, not every valid unfair omission. **Revision:** bounded finalized ordering windows, multi-path submission, authenticated receipts, rotating proposers, and fairness telemetry. No guaranteed censorship resistance or globally fair real-time ordering is claimed.

Threat model and security invariants

Adversary and trust boundaries

Assume modified clients, unlimited player identities, coordinated bots, malicious assets, arbitrary packet loss/reordering, partitions, hostile discovery responses, storage withholding, operator compromise, and manipulation of host clocks. Test infrastructure faults as adversarial events. Assume no honest majority among nearby players.

The proposed launch profile assumes a bounded Byzantine fraction among admitted validators in each authoritative committee, secure cryptographic primitives, uncompromised honest signing state, deterministic compliant runtimes, and enough retrievable state. These are assumptions to monitor, not consequences of a Merkle tree or Wasm sandbox.

Network liveness requires eventual sufficiently timely delivery and enough honest available validators. Deterministic consensus cannot promise unconditional termination in a fully asynchronous network with faults. Preserve safety by stopping canonical writes under uncertainty. [S11]

Required invariants

- **I1 - Authorized transition:** only the normative state machine can modify finalized state; all intents, capabilities, quantities, and object versions are checked.
- **I2 - Unique history:** no two honest nodes finalize conflicting successors under the stated committee fault bound and protocol rules.
- **I3 - Unique ownership:** one authoritative shard and version per live entity; no item is spendable in two shards or two offline branches.
- **I4 - Atomic effects:** a transfer has one durable decision; repeated delivery or crash recovery never reapplies its effect.
- **I5 - Conservation:** every material debit, credit, conversion, and sink balances in exact integer units, including rounding residues and migration.
- **I6 - Bounded work:** accepted messages and transitions have specified worst-case memory, execution, verification, output, and retention bounds.
- **I7 - Recoverability:** finalized state has retained data and verified recovery artifacts under a published failure model.
- **I8 - Epoch integrity:** keys, rules, membership, and checkpoints cannot be changed through discovery replies, client clocks, or a self-declared newer version.

Explicit exclusions and residual risks

No guarantee of humanness, bot exclusion, hidden public state, economic fairness, global anonymity, universal legal immunity, uninterrupted operation after arbitrary catastrophe, or resistance to compromise beyond the stated fault bounds. Honest operators must exist, communicate, retain data, and maintain software. An exhausted world can pause; absent hosts cannot execute agents.

Revised system and launch profile

Components and authority

Deterministic kernel: validates intents and computes state. Consensus operates on bounded input batches and certified resulting roots. Presentation, animation, audio, and speculative prediction are outside canonical execution.

Control journal: an internal BFT log for validator registry epochs, shard maps, ownership transfers, protocol migrations, checkpoint manifests, and transaction decisions. It is replicated across operators; there is no external public-chain dependency. Loss of its quorum prevents new control decisions.

Spatial shards: certified replicas store and execute region state. AOI/VON discovery and DHT routing locate peers but never authorize them. Halos contain read replicas of neighboring state, not independent writable copies.

Relays and transport: replaceable peer-operated routing, NAT traversal, and input distribution services. A relay cannot finalize state. If direct connections are impossible, independent rendezvous, STUN/TURN or relay infrastructure is required; serverless does not mean infrastructure-free.

Storage and archives: retain finalized snapshots, required replay logs, manifests, and authenticated coding fragments. Archives cannot vote unless independently admitted. An archive that stores data is not necessarily available to repair it.

Clients and agents: submit bounded signed intents and verify certified updates. They never nominate their own committee, alter resource counts, or assert elapsed world time.

Conservative launch configuration

- Illustrative control committee: 13 admitted independent operators, at most 4 Byzantine, commit quorum 9. Operator independence is reviewed outside gameplay; it is not inferred from IP addresses or key count.
- The same 13-operator authority handles initial shards. Scaling to separately sampled committees is disabled until committee capture, reconfiguration, and cross-shard safety models pass review.
- Use an existing reviewed BFT implementation and its exact deployment assumptions. This document does not specify a replacement consensus algorithm.
- Canonical offline rewards, executable user modules, unrestricted custom textures, unbounded avatars, and unbounded active terrain are disabled.

Tradeoff: this profile is a decentralized operator federation, not fully permissionless validation. It introduces governance and infrastructure costs. The source's stronger ownerless claim remains unestablished; it must not appear in product guarantees for this profile.

Consensus, membership, and availability

For an equal-vote committee of n = 3f + 1 certified validators, require q = 2f + 1 distinct authorized votes. Two q-sized sets intersect in at least f + 1 members. That is a necessary building block; safe finality also depends on the chosen algorithm's locks, phases, and commit rule. A root plus q signatures is not by itself a complete finality protocol. [S2, S3]

Required behavior

- Every proposal binds world ID, protocol epoch, committee epoch, shard ID, height/view, parent certificate, ordered input digest, resulting root, and availability commitment.
- Validators receive, validate, execute, and durably persist the entire bounded batch and resulting data needed for recovery before an eligible vote. An unexplained root cannot obtain an honest vote.
- Follow the selected protocol's actual commit rule. Distinguish a prepare/vote certificate from a commit certificate in types, storage, and client UI. No interchangeable certificate API.
- Persist locks, last vote, and epoch state before transmitting votes. Recovery after a disk failure cannot restore a snapshot that enables signing conflicts.
- Client certificate verification checks distinct signers, current authorized keys, exact epoch, parent lineage, and message domain. Relay signatures do not count as validator votes.
- Censorship detection records bounded authenticated submission receipts, not assertions of network-wide absence. Missing data or conflicting certified history triggers a safe pause and evidence retention.

Membership changes

Membership is not the set of currently connected avatars. The old committee finalizes a reconfiguration at a named height; the new committee installs that same state and acknowledges it. A protocol-specific, reviewed transition prevents old and new electorates from committing competing successors. During the transition, writes use the defined handover barrier. No independent local election or timeout reduces quorum.

Permanent loss of too many keys cannot be repaired merely by the remaining players declaring themselves validators. That requires the disclosed recovery/fork policy and a new world lineage.

Auditing and latency

Independent witnesses replay available logs and validate evidence; they improve detection but cannot guarantee censorship detection, availability, or prevention after majority capture. Fraud-based penalties require objectively checkable evidence and certified decisions. Do not retroactively rewrite an allegedly immutable history without a recovery policy.

Local prediction may target responsive frames. Finality latency is a separate measured distribution under actual network and operator conditions; no sub-50-ms global-finality claim is made.

Deterministic physics and state representation

Define one pure transition interface: Transition(previous state, ordered intents, certified context) -> next state, events, resource usage. The certified context contains only authenticated protocol data, including logical ticks and beacon values. All execution semantics are versioned and covered by golden vectors.

Numerical profile

- Q64.64 stores x as signed i128 X representing X / 2^64. Define legal ranges separately for position, velocity, force, mass, inventory quantity, and intermediate calculations.
- Multiplication computes an exact signed i256 product P = X * Y, then truncates P / 2^64 toward zero; the result must fit the target i128 and domain range. Negative arithmetic right shift is not automatically equivalent to truncation toward zero.
- Division computes trunc((X * 2^64) / Y) in a wide representation; Y = 0 is invalid. Specify integer square root, vector normalization, trigonometric approximations, and every tie-break. No undefined host math.
- Overflow, invalid conversions, or out-of-domain arithmetic reject the affected operation atomically under a specified rule. No partial writes, wrapping resource counters, or runtime-dependent saturation.
- Mass/resource quantities use exact smallest integer units where possible. Fixed-point recipes account for residues explicitly; repeated conversion cannot create a net positive quantity.

Execution profile

Ban floating-point opcodes and nondeterministic imports in the canonical module. Ban clocks, OS entropy, filesystem/network access, threads, shared memory, and runtime-dependent scheduling. Fix Wasm feature set, interpreter semantics, memory limits, instruction costs, and trap behavior. Native runtime security updates may preserve semantics but still require replay verification.

Canonical serialization fixes byte order, integer widths, lengths, enum tags, optional fields, and ordering. Reject alternate encodings, duplicate map keys, duplicate entity writes, and trailing bytes. Merkle leaf and internal-node hashes use distinct domain tags. Hashes commit complete canonical state, including ownership, nullifiers, pending transactions, and resource budgets.

Sort ECS entities/components explicitly. Specify collision and constraint order, nearest-object ties, terrain/noise algorithms, and PRNG streams. Parallel optimization must produce the same normative result.

Verification required: identical roots on x86-64, ARM64, and the chosen proving VM for random and edge-case sequences. Formal verification claims must name the exact property, model, code relation, tool version, and remaining assumptions.

Input authorization, ordering, and rollback

Canonical action envelope

Every intent MUST bind a domain tag; world and protocol epoch; authority shard and ownership version; actor ID and key epoch; capability/session ID; monotonically tracked sequence; target logical tick; action type; bounded canonical payload; and signature. IDs and hashes are fixed size. Limits are enforced before allocation or signature verification.

The state machine checks authority, active session, sequence, allowed tick window, cooldown, target object versions, capability, and resource cost. A signature authenticates bytes; it does not establish that the signer is entitled to perform the action. Session capabilities have action scope, inventory/spending bounds, and finalized revocation state.

Proposed baseline ordering

The committee finalizes an ordered batch for each logical tick. Within a tick, the normative order is actor ID then sequence, with explicitly defined system events and cross-shard decisions. New player IDs are assigned by certified creation records; player-chosen key grinding cannot choose processing priority. This rule is deterministic but is not a claim of fair real-world arrival ordering.

Permit at most one eligible state-changing intent per actor per tick in the initial profile. Higher rates need a separate budget and scheduling model. A missing sequence cannot create an unbounded wait: the committed batch contains explicit deterministic no-op/skip handling. Late packets outside the finalized window are rejected, never inserted into finalized history.

Speculation stays reversible

Clients may predict movement and render speculative outcomes in a bounded rollback buffer. Finalized combat, trade, inventory, death, and transfer effects cannot be overwritten by a late input. Speculative assets cannot be exported, sold, or used as canonical collateral. Display finality status where it affects player choices.

Cap rollback ticks, resimulation fuel, pending sequence gaps, and input backlog. Do not trust a player's local timestamp or award rewards from time spent disconnected. A committed tick advances the world; a host clock is diagnostic only.

Replay and fairness limits

Retries are idempotent, reconnects use a new finalized session/key epoch, and actor sequence state survives shard migration and snapshot recovery. Transaction IDs are consumed once. Closed-epoch rejection and retained per-actor/session high-water marks permit bounded deduplication: pruning a tombstone MUST NOT make an old action valid again. Pending decisions stay retained until completion is certified; their state cannot be pruned by age alone. Multi-path input submission and proposer rotation reduce single-relay dependence, but validators may still censor within the declared fault model. Measure omissions and delay; do not promise perfect simultaneous combat ordering across the Internet.

Cross-shard transactions and unique ownership

Every entity has one authority shard, an ownership version, a canonical value, and a lock/transaction status. Halo replicas are read-only. Trades, inventory movement, resource claims, agent host transfer, and avatar migration use the same versioned transaction machinery.

Durable decision protocol

1. **Declare:** finalize a transaction ID and immutable digest in the control journal. The declaration includes all participant shards, exact read/write versions, maximum footprint, and expiry in control-journal heights.

2. **Prepare:** each participant checks the complete digest and expected versions, locks the exact write set, persists its prepare record through shard consensus, and returns a certified prepare. Conflicting operations cannot use a locked object.

3. **Decide:** the control journal verifies all prepares and records one COMMIT or ABORT. COMMIT is allowed only with matching, unexpired valid prepares from every participant. Expiry and decision ordering are specified in the same journal; decision is write-once.

4. **Apply:** participants verify the final decision certificate, apply it exactly once, advance object versions, and persist the consumed transaction ID. Readers recognize prepared/pending objects until decision application completes.

5. **Recover:** any peer can fetch and relay the durable decision. Local receipt time, coordinator failure, or network timeout cannot independently unlock a prepared object.

This is a proposed BFT-backed transaction construction, not an established proof of atomicity. Its complete state machine, consensus integration, expiry races, and recovery behavior MUST be modeled before implementation. Classical two-phase commit can block when its coordinator fails; consensus-backed decisions must still satisfy their own availability assumptions. [S7]

Blocking and concurrency

A control-journal outage may leave locks pending. Show that state to clients; do not unlock for convenience. Bound outstanding prepares, participant count, lock duration measured in finalized heights, and locked resources per admitted budget. Use a canonical lock-acquisition order; incomplete preparation expires through an ABORT decision in the live journal. Transactions can be rejected before prepare to avoid unbounded congestion. Require complete, immutable read/write sets; no surprise participant discovery after locks.

Migration consumes the old authority/version and creates one new version at the destination. The avatar is not spendable or authoritative in both places during handover. Atomic visibility is defined through certified decision state; unrelated regional snapshots are not a valid cross-shard read. The control journal is an intentional scaling bottleneck at launch, to be measured rather than hidden.

Spatial seams, collision authority, and shard churn

AOI subscriptions control transmission volume; they do not define ownership or the full set of physical interactions. Each projectile, entity, and irreversible event has one certified authority and event ID. Neighboring shards can predict/read an interaction but cannot independently issue a second authoritative effect.

Replace the fixed halo with a validated bound

For a maximum interaction delay/rollback horizon T, a simple coverage requirement is:

halo >= interaction radius + v_max * T + 0.5 * a_max * T^2 + numerical margin

This bound assumes speed and acceleration limits are actually enforced and uses consistent units and deterministic conservative rounding. It is not sufficient for instantaneous teleportation, arbitrary-area actions, or attacks exceeding those bounds; those actions require explicit remote participation. Swept geometry, not only endpoint positions, determines crossing and collisions.

For an interaction that changes objects in multiple shards, collect certified snapshots at an agreed logical barrier and use the transaction procedure on page 13. The chosen action specification defines who evaluates the collision, which object versions it uses, and when damage becomes irreversible. A source shard cannot claim a remote target accepted damage without authenticated destination processing.

Failure semantics

- If required neighbor data or quorum is unavailable, block the irreversible interaction or keep it visibly speculative. Do not let the initiating client select a fallback hit outcome.
- Deduplicate event IDs across halo replication, relays, retries, migration, and replay. Terrain extraction has globally unique resource coordinates and versions.
- Define border position ties, chunk-coordinate conversion, projectile expiry, negative coordinates, and ownership when an entity spans several volumes.

Topology changes

Split/merge maps are finalized in the control journal at a named barrier. Retain old routing aliases and replay IDs through a bounded migration interval. Install the full state, ownership map, pending decisions, deduplication state, and relevant logs before new authorities can write. Two child shards cannot each inherit a spendable copy of one object.

Use upper/lower load thresholds, minimum shard lifetime, and a maximum transition rate. Topology churn cannot increase a region's certified entity, work, or storage budget. Collocated crowds may still exceed capacity: cap interaction fanout, queue or reject new activity, and measure actual byte/CPU costs. Neither octrees nor relay aggregation removes the cost of densely interacting actors.

Isolation and proof-based execution

Safe launch behavior

Disconnected or unwitnessed play is a local sandbox branch. It can visualize deterministic terrain and rehearse actions, but creates no canonical reward, inventory, damage, resource extraction, authority migration, or elapsed-time credit. Reconnection restores the latest authenticated canonical state. An empty region may still be validated remotely by admitted operators; geography does not determine committee existence.

Conditions for a future canonical offline mode

Before isolation, finalize an exclusive lease covering exact entities, inventory, resource cells, starting root, logical tick range, budget, and authority epoch. Canonical writers cannot modify leased state. Reconciliation consumes the lease once in the control journal and atomically installs one permitted successor. Conflicting exits are rejected even when both proofs are valid.

Define lease expiry and return races without host clocks. An expired lease cannot later merge valuable effects. Canonical reclamation and late proof submission must refer to the same authoritative decision. Offline replay cannot fight live entities, touch unleased resources, rewind failed attempts, or choose a favorable random branch.

Proof claim and trust boundary

Bind the proof to the expected program/interpreter ID, verifier/proof-system version, world and epoch, lease ID, start/end roots, ordered input commitment, tick range, resource budget, beacon transcript, and unique exit record. Verify every output against the intended lease and current canonical lineage. Execution validity alone does not prove real elapsed time, honest input capture, availability, or that another valid branch does not exist.

RISC Zero uses a compiled guest program identity and public journal; this does not automatically certify a Wasm physics kernel. Specify either a deterministic Wasm interpreter inside the guest with its exact hash and semantics, or a separately compiled reference transition implementation with an equivalence argument. [S4]

Production verification MUST reject fake/development receipts, unexpected proof types, unresolved assumptions, wrong program IDs, malformed public outputs, stale epochs, and over-budget witnesses. Pin the actual soundness/setup assumptions; a wrapper may introduce additional trusted setup or cryptographic requirements. [S9]

Release blocker: leases, proof binding, reconciliation, VM equivalence, proving cost, and conflicting-proof tests require independent review. Until those pass, offline canonical progression stays disabled. No mobile proving or millisecond-verification claim is made without measurements on representative hardware.

Randomness without selective disclosure

Pairwise player commit-reveal is removed from consequential outcomes. The last participant can withhold a reveal after observing an unfavorable combined value. A penalty can deter behavior but cannot make disclosure inevitable. Never describe that problem as requiring a cryptographic collision.

Proposed internal threshold beacon

Admitted operators jointly run a reviewed threshold randomness scheme with authenticated distributed key generation, agreement on one public-key/transcript identity, an explicit share threshold, and secure share custody. A drand-style design illustrates the required setup and failure assumptions; using it does not eliminate operator trust or DoS. This profile proposes an internal beacon, not reliance on an external service. [S12, S13]

Each random action batch is finalized first, with its irrevocable action costs and a designated future beacon round. Operators MUST NOT release valid shares for that designated round before the action batch has a finality certificate. Multiple actions may share one round using distinct fixed event IDs. The protocol must define a gating mechanism consistent with the selected beacon scheme; a schedule that releases results before commitments is invalid.

The event output is H(domain, world ID, rules epoch, beacon identity, round, verified beacon output, finalized event ID). Commit the event ID and resource version before entropy release. Map outputs to probability ranges by specified rejection sampling, not biased modulo reduction. Retries cannot change IDs, action commitments, costs, or rounds.

Withholding and outages

An honest available share threshold is required for progress; an adversary below the beacon compromise threshold must not predict the gated result before action commitment. Publish exact thresholds and joint fault assumptions with the consensus committee. DKG failure, share theft, or delayed release invalidates those assumptions.

If the designated beacon is missing, the outcome remains pending. Do not use a player's nonce, host clock, previous round, or new selectable round as fallback. Do not refund and re-open a committed valuable attempt merely because a client disconnects. Limit pending random actions and reserve resources before accepting them.

Delay functions and public terrain

A VDF gives delayed, verifiable evaluation of a fixed input; it does not create fresh entropy. Input bias, retries, and evaluation-speed differences still require analysis. [S5]

Procedural terrain derived from a public seed is predictable. Valuable secrets cannot be protected by hiding the UI. Fully unpredictable ongoing randomness is unavailable when no honest beacon operators remain.

Transport, discovery, and connection defense

Authenticate authority; distrust routing

DHT records bind world, namespace, shard, certified authority epoch, operator key, permitted addresses, record sequence, and expiry. Validate signatures and the current registry against a pinned checkpoint. Routing entries are suggestions; a close XOR distance does not confer legitimacy. Invalid claims never update consensus membership or shard maps.

Join through several independently operated bootstrap paths and compare certified lineage. Retain last-known-good peers and checkpoints. Diversity by IP prefix, provider, region, and operator improves resilience but does not prove distinct control. Committee or sentinel selection from freely generated DHT keys is forbidden.

Connection and ingress controls

- Use authenticated encrypted transport with peer identity bound to certified keys and world/protocol context. Pin allowed protocol versions; do not accept attacker-directed downgrades.
- Enforce QUIC address validation and anti-amplification behavior, including the protocol's three-times response limit before validation. Disable application state changes through replayable early data. [S14]
- Cap handshakes, concurrent streams, message lengths, pending fragments, queue depth, idle sessions, and aggregate work per node. Use cheap structural checks and admission budgets before expensive signatures, proofs, or asset processing.
- Layer per-connection and per-actor limits with a global region/node budget. An attacker using many player identities must not multiply total available compute without bound.
- Separate consensus, control, gameplay, audit, and bulk recovery queues. Reserve capacity for certified recovery/control traffic without allowing forged priority markers to bypass limits.
- Bound neighbor fanout and gossip duplicates with message IDs, hop limits, and finite caches. Reject unsolicited bulk snapshots or proofs.

Address and privacy boundaries

Automatic dialing cannot target loopback, link-local, local administration endpoints, or arbitrary internal networks through player advertisements. Private-network support requires explicit operator configuration. Prevent address-parser ambiguity and DNS rebinding where names are allowed.

Public P2P connections expose addresses, location-related behavior, and timing. Relays can reduce direct exposure but introduce cost, availability, and observer trust. Do not promise anonymity. Minimize long-lived telemetry and avoid real-world identifiers in replicated state.

Residual risk: sufficient bandwidth or infrastructure compromise can still cause denial of service. Rate limits protect capacity; they do not ensure unlimited honest participation or continuous global reachability.

Runtime isolation and real resource budgets

The trusted computing base includes consensus code, canonical serializers, signature/proof verifiers, arithmetic, state storage, the Wasm runtime, host bindings, compiler/toolchain, and update delivery. A formally checked kernel does not automatically verify these other layers. Runtime vulnerabilities and compiler faults remain relevant. [S8]

Launch policy

Only the pinned, reviewed canonical kernel executes on validators. Player-defined executable modules are disabled. Built-in actions, assets, and recipes have deterministic bounded complexity. Off-consensus planners cannot call consensus host functions directly.

Required sandbox controls

No WASI filesystem, network, environment, clock, or entropy imports. Allow only documented pure host calls with bounded arguments/results. Cap linear memory, tables, stack depth, module size, instruction fuel, entities touched, collision pairs, emitted events, and output bytes. Prohibit shared memory and threads in canonical execution.

Use a separate low-privilege process for untrusted decoding, optional asset conversion, proof processing, and rendering where practical. Deny network and filesystem access by default. Do not deserialize native compiled/AOT artifacts from peers. Resource ceilings apply to host calls and verifier work as well as Wasm instructions.

Deterministic out-of-fuel or invalid-input behavior is specified. A transaction computes against an isolated write set and commits atomically; a trap cannot leave an inventory debit without its corresponding credit. Operational deadlines can stop work, but cannot select a different canonical result based on which machine was slower.

Mandatory admission envelope

Before network activation, publish immutable per-epoch maxima for encoded and decoded message bytes, nested depth, input rate, batch fuel, state growth, proof/witness bytes, verification work, asset complexity, pending transactions, and bulk-download bandwidth. Publish tested capacity on the minimum supported validator hardware and reserve operational headroom.

Do not invent universal safe numeric values in an architecture document. Default configuration is deny-unconfigured: nodes MUST refuse to start if a required limit is absent, if batches can exceed the configured machine capacity, or if the limits disagree across the epoch. Benchmark before raising a limit and certify changes.

Fuzz decoder/runtime boundaries and isolate crashes. A malicious batch must not cause every replica to restart repeatedly on replay. A confirmed runtime bug requires a safe halt and patched epoch; no permanent freeze can substitute for incident response.

Content policy and bounded carrier capacity

The source's proposed 2,048-vertex and 128 x 128 texture caps reduce volume but do not remove covert channels. An indexed texture with p possible colors and d independently selectable pixels has up to d * log2(p) bits of representational capacity before normalization. If more than one accepted output remains, those choices can encode information. Geometry, placement, timing, and keys add channels.

Revised baseline

Use only built-in vetted mesh/material IDs, bounded construction primitives, and short schema-constrained metadata. Omit arbitrary textures, scripts, external links, embedded files, custom shaders, archives, and opaque application blobs at launch. Fixed-size cryptographic objects remain necessary but do not create a guarantee of zero information carriage.

Player construction remains expressive within finite entity count, quantized coordinates, bounded region footprint, and real update/storage budgets. Names or other optional text need bounded length, normalization, display escaping, and a retention/moderation policy. The protocol cannot guarantee a socially useful world will never be used to communicate or trade.

If custom visual assets are enabled later

- Canonicalize every accepted attribute, ordering, coordinate range, material reference, and palette; reject metadata not explicitly allowed.
- Disallow non-finite/unbounded values, excessive degenerate triangles, self-referential structures, invalid indices, and collision geometry inconsistent with allowed mechanics.
- Convert through an isolated, metered implementation; run differential canonical-output tests. Validators cannot disagree about whether input is valid or what canonical asset it creates.
- Bound both total upload volume and distinct accepted outputs. Deduplicate canonical objects, charge actual retained bytes, and expire optional caches under a published policy.
- Test client decode/render paths independently from consensus conversion. Small assets can still be malicious.

Retention and operator choice

Operators can choose not to cache optional user content, provided they still satisfy their declared validator/storage obligations. Finalized hashes do not require every peer to retain every historical asset forever. Specify what is canonical state versus optional archival presentation before making durability claims.

Encryption and erasure coding do not remove responsibility or guarantee immunity; this document makes no legal conclusion. Operator policy and jurisdiction-specific review remain necessary. The correct technical objective is bounded abuse surface, not impossible arbitrary data hosting or perfect steganography destruction.

Storage, checkpoints, and authenticated coding

What must survive

A genesis seed regenerates unmodified terrain only when the exact generator, rules, and parameters survive. It does not regenerate harvested resources, buildings, ownership, consumed transactions, progression, or historical player choices. Archive the kernel, toolchain, formats, test vectors, and migration lineage with the seed.

Define two contracts separately: **world continuity** requires a certified current snapshot and the retained log after it; **full historical replay** requires all referenced historical data. Finite mandatory retention can support continuity but cannot promise every past action remains available forever.

Finalization and checkpointing

Validators durably retain accepted batches and data before votes. Periodic checkpoints include the complete state, ownership map, pending transactions/leases, deduplication state, resource counters, shard map, membership lineage, and cryptographic suite IDs. The control journal certifies the manifest and names the replay range.

Replicate snapshots across independently operated storage domains; include at least one complete independently restorable copy in the operational plan. Retain old recovery material until a new checkpoint has passed independent retrieval/reconstruction and lineage verification. Log pruning follows a certified retention barrier, not local disk pressure.

Erasure coding

Split objects into bounded source blocks. Manifests fix object hash, length, codec/version, symbol size, source-block parameters, and authenticated fragment commitments. Duplicated symbols do not add recovery information. Collect enough distinct, valid, mathematically sufficient symbols for every required block. RaptorQ's documented probabilistic decoding behavior is not a claim that every chosen K + epsilon set is sufficient. [S6]

Authenticate fragments before expensive decode using certified commitments or authenticated source data. For dynamically generated repair symbols, require a verified source/commitment mechanism; a hash announced solely by the serving peer is not enough. Always verify the final reconstructed object against the certified manifest. Bound decoder memory, retries, CPU, and output size to prevent coding/manifest pollution attacks.

Availability measurement

Audit random retrieval from independent operators, actual failure domains, symbol coverage, repair throughput, and complete restore time. A valid proof of possession or a successful challenge at one moment does not guarantee future delivery. Replenish replicas before safety margins disappear; if new writes cannot meet the retention contract, pause admission. In-game upkeep cannot purchase storage unless real operators undertake and fund it.

Catastrophic attrition and recovery policy

Correct the survival claim

Let D be incompressible required recovery data, N the peer count, and s the number of arbitrary surviving peers. If every s-peer subset must restore D, the sum of information stored by every such subset must be at least D. Averaging over subsets implies total stored information is at least (N / s) * D, even before authentication, metadata, and practical coding overhead.

For N = 100,000 and s = 100, a universal any-survivors guarantee therefore requires at least 1,000 times D in aggregate information. This is a necessary lower bound, not a sufficient construction, and sufficient rank/retrievability is required for every block. If all copies of one necessary block are lost, no fountain code can recreate its unknown contents.

Under a much weaker independent random-node-survival model with survival probability p and r complete copies of one object, the probability all copies are lost is (1 - p)^r. At p = 0.001, reaching a per-object loss probability at most 10^-6 needs r >= ceil(log(10^-6) / log(0.999)) = 13,809 independent copies. Correlated outages invalidate that estimate; many objects need a world-level failure budget. This example illustrates redundancy cost, not a proposed placement scheme.

Recovery procedure

1. Obtain an explicitly trusted checkpoint or authenticate lineage to a previously pinned one. Compare independently operated archives; neither the highest claimed height nor DHT proximity decides authenticity.

2. Fetch and verify manifests, membership certificates, codec parameters, and required kernel/rules artifacts.

3. Restore every required state block and verify object hashes and the complete snapshot root before publishing state. Missing blocks remain visibly unavailable.

4. Replay the certified retained suffix deterministically; restore pending decisions, locks, sequence/nullifier state, and voting state without enabling double-signing.

5. Resume only with the certified current committee and required quorum. Durable data recovery does not recreate lost validator keys or authorization.

Total quorum loss

If canonical membership can no longer authorize progress, surviving operators may create an explicitly labeled recovery fork from the last verifiable checkpoint. It receives a new world ID and declared trust assumptions; users choose whether to follow. This is not proof that the original world continued without interruption or historical loss.

Publish the actual failure model, measured restore results, retained history scope, and funding obligations. Replace the source's 0.1% guarantee with these conditional commitments.

Agents, economy, and world time

Agent authority and execution

Bots and human-controlled clients use the same authorized action interface and physical limits. The system does not infer humanness from a signature. An autonomous planner runs outside consensus with a scoped session capability: allowed actions, maximum spending, exact owned entities, sequence, key epoch, and revocation rules.

A planner may be compromised or manipulated by untrusted content. It never holds validator keys, update authority, archive credentials, arbitrary shell/network access granted by the protocol, or unlimited inventory permissions. Transactions and trading must honor capability limits independently of planner instructions.

An agent's durable identity and assets survive in canonical state; its cognition does not run without a host. Optional volunteer hosting requires a finalized exclusive host-control lease, bounded workload, failover policy, and versioned transfer. Two hosts cannot both spend one agent inventory. If no host remains, behavior pauses or follows a specified deterministic dormancy rule.

Economic invariants

Resource cells have unique world coordinates, canonical remaining amounts, and versions. Procedural regeneration uses the seed plus canonical extraction state; revisiting or migrating cannot reset depleted matter. Bound the active world and new-region creation rate: infinitely expanding procedural geography is not a closed finite economy.

Track exact integer units for each conserved material, production input/output, emissions, waste, and explicitly modeled source or sink. Rounding residues belong to a deterministic ledger; repeated craft/recycle/trade loops cannot generate net value in conserved units. Reject negative amounts, overdraws, counter overflow, and recipe cycles that violate intended bounds.

Conserved mass does not imply conserved economic value. Scarcity, monopolies, hoarding, inactive accounts, and optimal bot strategies can still distort markets. Simulate adversarial strategies and impose mechanism-specific limits where gameplay requires them. Do not promise autonomous competition always sustains a healthy town.

Time and actual capacity

Decay, cooldowns, rewards, and agent upkeep use finalized logical ticks. If canonical progress stops, the default world clock stops. No participant can fast-forward it by reporting elapsed wall time. Any catch-up policy must be separately specified, bounded, certified, and tested for supply/decay effects. Logical ticks do not prove physical elapsed time: real-time cadence requires disclosed clock/rate assumptions and operational monitoring. No globally exact wall-clock pacing guarantee is made.

In-world food and rent are game mechanics. Real CPU, storage, egress, repair, and host incentives require separate admission caps and operating commitments. Valid, well-fed agents can still exhaust infrastructure without those controls.

Key lifecycle, upgrades, and long-range defense

Key separation

Player recovery/root keys, scoped session keys, validator voting keys, beacon shares, release-signing keys, and storage-service credentials are distinct. Enforce strict vetted signature decoding and verification; do not assume example cryptographic code is suitable for production. Bind signed objects to their intended domain and epoch. [S10]

Use secure randomness for key generation, encrypted backup, and hardware-backed custody for operator keys where practical. Player recovery options - user-controlled backup or preconfigured multi-party recovery - must disclose trust and loss consequences. A new person claiming an old name cannot recover assets without the configured authorization.

Operator state and evidence

Persist last signed view/height, lock certificate, epoch, and pending transitions before sending signatures. Disk snapshot rollback or cloned validator instances cannot resume signing independently. A validator recovering lost signing state requires a reviewed rejoin path, not an empty database.

Key rotation is finalized before activation. Retired keys cannot authorize new epochs; preserve authentic verification material for historical records. Evidence of equivocation binds the same signer, epoch, and signing domain to incompatible messages. Evidence of invalid execution requires data sufficient for deterministic verification. Mere non-response, unproven accusation, or receipt of one unavailable root is not proof of cheating.

Protocol maintenance

Keep stable world semantics where possible, but permit necessary security repairs. Each migration specifies code/rules hashes, activation barrier, affected state transformation, compatibility and invariant tests, quorum authorization, and client consent/upgrade behavior. Operational security fixes preserving semantics are still replay-tested.

Publish signed releases through independent channels, reproducible build instructions, SBOMs, locked dependencies, archival toolchains, and independent build attestations. Stage updates before activation; avoid single-maintainer authority and self-update from unauthenticated discovery records. Incident response can halt new commits, preserve evidence, and coordinate an explicit repaired epoch or recovery fork.

Century horizon

No current signature, hash, proof system, runtime, operator set, or archive is guaranteed secure and available for a century. Version cryptographic suites and migrate commitments while existing authentication remains trustworthy. Preserve history through chained, independently archived attestations; do not claim a later re-signature magically authenticates an already compromised past.

Long-offline clients need a trusted recent checkpoint or explicit recovery choice. Key erasure/evolution and operator rotation reduce risks but do not replace authenticated lineage. Stewardship, funding, and recovery are part of longevity.

Verification matrix | Canonical integrity

The following are required acceptance scenarios. They have NOT been executed in this review. Model invariants I1-I8 from page 8, then test the same properties in the implementation under crashes, partitions, and malicious messages.

T01 | Committee capture and membership | F01, F02, F10, F28

Create unlimited player keys and grind DHT proximity. None gains validator authority. Partition a 13-member committee into incompatible connected subsets; no conflicting commits occur within the stated f = 4 bound. Membership handover cannot allow old/new split-brain finality. Test forged registries and old-key histories.

T02 | Persistence and certificates | F02, F07, F27

Crash immediately before/after vote persistence, certificate receipt, and commit application. Replay old disk snapshots, clone a voting instance, duplicate signers, mix epochs, and submit prepare certificates as commits. Reject invalid finality; recovery never double-signs. Withhold data and verify honest votes are refused.

T03 | Transfer safety | F03, F04, F12, F17

Model every PREPARE/COMMIT/ABORT/expiry ordering. Crash each participant and decision replica at each durable boundary. Retry delivery and race two spends/migrations. Exactly one ownership version is spendable; all effects apply once. Loss of decision quorum leaves explicit pending locks, never unilateral release.

T04 | Offline claims | F03, F06, F18

At launch, no offline branch affects canonical state. For a future lease mode, submit two individually valid proofs from the same lease to different shards, a post-expiry exit, wrong program ID, wrong start root, changed beacon transcript, fake receipt, and unresolved proof assumptions. At most one authorized exit is accepted.

T05 | Determinism and conservation | F13, F14, F26

Run identical long randomized input sequences on x86-64, ARM64, and the chosen proving interpreter/guest. Require bit-identical roots. Cover signed extrema, negative rounding, divide-by-zero, zero vectors, object ties, malformed serializations, terrain borders, recipe loops, and residue accounting. Every fault is atomic and deterministic.

T06 | Spatial seams and topology | F11, F23

Fire maximum-speed projectiles across several borders; apply large-radius effects and movement at exact boundaries. Partition a neighbor; duplicate halo deliveries; split/merge with pending transfers; oscillate entities across borders. No duplicate/missed canonical effect, unauthorized remote write, or two-child inventory copy occurs.

T07 | Randomness and ordering | F05, F06, F30

Withhold reveals/shares, attempt early beacon release, reorder or omit actions, disconnect after learning an outcome, grind keys/event IDs, and replay attempts under different rounds. Commitments precede gated entropy; a missing result stays pending; no client selects a substitute outcome or refunded reroll.

Verification matrix | Capacity and release gates

T08 | Parsing, sandboxing, and proofs | F15-F18, F21

Fuzz lengths, offsets, nesting, duplicate fields, signatures, proofs, meshes, and compressed data. Exercise fuel exhaustion, memory growth, traps, proof verification floods, and pathological collision pairs. Bound allocation/work before admission; no crash loop or partial state persists. Differential decoders must agree.

T09 | Discovery, transport, and privacy | F10, F24, F29

Eclipse joiners, poison addresses, forge versions, replay handshakes/early data, exhaust connections, and target internal addresses. Clients require authenticated lineage; state-changing early data is refused; ingress stays within capacity. Document IP/timing exposure and public-state visibility instead of falsely passing a secrecy test.

T10 | Content and persistence | F19-F23

Attempt bit encoding through every permitted choice and measure residual capacity. Submit maximum valid edits, agents, and split triggers through many identities. Actual global memory/CPU/egress and retained bytes remain bounded. No claim that a passed sanitation test proves zero covert channels.

T11 | Restore and coding pollution | F07, F08, F20, F22, F28

Restore from checkpoint plus suffix with complete, partial, duplicated, corrupted, missing, and forged symbols/manifests. Remove correlated operator/provider groups and targeted blocks. Verify data before execution; missing state blocks prevent false recovery. Test archived kernel/toolchain availability, locks/nullifiers, lost keys, and an explicitly new recovery fork.

T12 | Agents, keys, and updates | F25-F30

Compromise an agent planner, race hosts, steal/revoke sessions, roll back a validator disk, forge penalty evidence, and deliver a malicious update or old suite. Cap spending and authority, preserve evidence, and refuse unauthorized code/epoch changes. Test outage-time decay, dormancy, and bounded catch-up policy.

Mandatory launch gates

- A complete state-machine spec, including epochs, transaction expiry, input windows, budget enforcement, and failure semantics, exists with no security-critical behavior left to client discretion.
- Model checking finds no counterexample to I2-I4 in the modeled fault bounds and reconfiguration/recovery cases. Proof scope and limitations are published.
- T01-T12 pass with preserved artifacts and reproducible fixtures. Differential replay and meaningful coverage-guided fuzzing cover trust boundaries.
- Measured p50/p95/p99 latency, finality, throughput, restore time, proving/verifying cost, and attack capacity meet the published minimum-hardware profile. No marketing latency is substituted for measurement.
- Independent cryptographic/protocol and implementation reviews address findings; critical/high defects are fixed and regression-tested. Residual risks have named operator responsibility and an incident/fork plan.

Passing these gates supports a bounded assurance claim. It still does not prove absence of all exploits. Reassess after code, dependencies, topology, mechanics, keys, or operational assumptions change.

Replacement implementation directive

Build Aetheria incrementally as an experimental deterministic, peer-operated world. Do not describe the result as production-grade, cheat-proof, ownerless, permanently safe, or catastrophe-proof until evidence supports each precise claim. The source document's implementation prompt is superseded by this page and the mandatory rules in this revision.

Stage 1 | Executable specification

Write normative schemas, domains, arithmetic, canonical serialization, actor/object authority, resource ledgers, deterministic ordering, budgets, and rejection semantics. Provide a reference transition implementation and golden vectors. Implement no networking, arbitrary modules, or canonical offline progression yet.

Deliver tests for overflow, rounding, conservation, replay, traps, and deterministic execution. Separate human-readable rules from their executable implementation and document their relation.

Stage 2 | Certified single-authority network

Integrate a reviewed BFT implementation with the fixed admitted committee, durable signing state, authenticated proposals, complete data retention, checkpoint lineage, and partition behavior. Build peer discovery and relays as untrusted routing around that authority. Deliver reproducible fault-injection and crash-recovery results.

Stage 3 | Shards and durable transactions

Specify and model unique ownership, control-journal decisions, prepare/expiry races, idempotent application, spatial collision authority, and split/merge handover. Activate only after T01-T03 and T06 pass. Benchmark control-journal capacity before claiming large-world scalability.

Stage 4 | Storage and operations

Implement bounded snapshots/log retention, authenticated coding, independent placement, repair audits, full restoration, archive lineage, key custody, signed reproducible releases, and incident response. Publish the actual survival model and real infrastructure obligations.

Stage 5 | Separately gated research extensions

Only after independent review, consider threshold-random gameplay, canonical leased offline execution, executable user content, confidential state, resource-based permissionless admission, and distinct sampled shard committees. Each changes the threat model and requires its own specification, measurements, and regression scenarios.

Required deliverables

An architecture and protocol spec; reference implementation; golden vectors; formal models with properties and counterexample history; adversarial tests; benchmark fixtures/results; cryptographic assumptions; SBOM/reproducible build; key/backup/restore runbooks; and a findings ledger mapping F01-F30 to code, tests, evidence, and remaining limitations.

Rust is appropriate for memory-safe implementation, but reviewed libraries and narrow unsafe boundaries matter more than language choice. No cryptographic primitive, consensus algorithm, or prover should be invented merely to satisfy a code-generation prompt.

Evidence and primary references | 1 of 2

The source architecture was reviewed in full. Original page references in F01-F30 preserve traceability after this rewrite. The findings are this review's architectural analysis; sources below establish underlying mechanisms or limits, not a proof that the revised construction is secure.

The source bibliography contains 42 entries, including secondary articles, discussion threads, indexes, and sources cited for claims beyond their demonstrated scope. This review does not certify every original entry. Unverified source claims are not retained as guarantees. In particular, the RaptorQ reconstruction claim is checked directly against RFC 6330, not inferred from a decentralized-storage whitepaper; legal immunity and formal kernel verification require evidence the source did not supply.

S1 | Identity and Sybil resistance

John R. Douceur, The Sybil Attack, IPTPS, 2002. Establishes the identity-multiplication problem and limits of decentralized redundancy without strong identity/resource assumptions. Does not validate Proof-of-Gameplay Stake. https://www.microsoft.com/en-us/research/publication/the-sybil-attack/

S2 | Byzantine replication

Miguel Castro and Barbara Liskov, Practical Byzantine Fault Tolerance, OSDI, 1999. Baseline primary work on replicated execution under Byzantine faults. A reference to BFT is not an implementation of its protocol.
https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance

S3 | Consensus assumptions and performance

Maofan Yin et al., HotStuff: BFT Consensus in the Lens of Blockchain, arXiv:1803.05069, revision 6, 2019. Leader-based partially synchronous BFT; linear communication claims depend on its protocol and model. Does not justify the source's unspecified leaderless local finality.
https://arxiv.org/abs/1803.05069

S4 | Execution proofs and program identity

RISC Zero, zkVM Overview, official developer documentation, version 3.0, accessed 8 October 2026. Describes guest program identity, receipts, public outputs, and proving workflow. Does not establish ownership, real elapsed time, or Wasm equivalence. https://dev.risczero.com/api/zkvm

S5 | Delay functions

Dan Boneh, Joseph Bonneau, Benedikt Bunz, Ben Fisch, Verifiable Delay Functions, CRYPTO 2018; IACR ePrint 2018/601. Defines delayed sequential evaluation with efficient verification. Entropy and action commitment remain separate application concerns. https://eprint.iacr.org/2018/601

S6 | Fountain coding and corruption

IETF RFC 6330, RaptorQ Forward Error Correction Scheme for Object Delivery, 2011, especially sections 5.8 and 6. Defines sufficient-symbol recovery behavior and coding-related corruption concerns. Does not guarantee arbitrary node-loss survival or permanent fragment availability.
https://www.rfc-editor.org/rfc/rfc6330

Evidence and primary references | 2 of 2

S7 | Transaction decision failure

Jim Gray and Leslie Lamport, Consensus on Transaction Commit, ACM TODS 31(1), 2006; arXiv:cs/0408036. Classical two-phase commit can block; consensus-backed decision replication has its own fault model. This is not a proof of the proposed BFT transfer design. https://arxiv.org/abs/cs/0408036

S8 | Runtime boundary

Wasmtime, Security, official documentation, accessed 8 October 2026. Documents the sandbox security boundary and threat model. A sandbox does not imply a flawless implementation, deterministic application, or universal resource bound. https://docs.wasmtime.dev/security.html

S9 | Production proof acceptance

RISC Zero, Proof System Overview, official documentation, accessed 8 October 2026. Describes receipt/journal structure and receipt variants, including test/fake receipts. The actual verifier configuration and proof assumptions must be pinned. https://dev.risczero.com/proof-system/

S10 | Signature implementation

IETF RFC 8032, Edwards-Curve Digital Signature Algorithm (EdDSA), 2017. Algorithm and verification reference. Does not provide account recovery, operator independence, or secure key custody. https://www.rfc-editor.org/rfc/rfc8032.html

S11 | Asynchronous termination limit

Michael J. Fischer, Nancy A. Lynch, Michael S. Paterson, Impossibility of Distributed Consensus with One Faulty Process, MIT-LCS-TR-282, 1982; JACM, 1985. Rules out unconditional deterministic termination in the modeled fully asynchronous fault setting. https://dspace.mit.edu/entities/publication/287e7d14-a6f3-4ab4-b21a-a98ea86c1212

S12 | Threshold beacon mechanics

drand, Protocol Specification, official documentation, accessed 8 October 2026. Defines group identity, DKG/resharing, threshold signatures, and beacon verification. An internal adapted design still requires agreement on setup and application gating. https://docs.drand.love/docs/specification/

S13 | Beacon failure assumptions

drand, Security Model, official documentation, accessed 8 October 2026. Discusses share compromise, withholding, DoS, and DKG failure assumptions. A beacon has availability and secrecy conditions; it does not prevent action censorship. https://docs.drand.love/docs/security-model/

S14 | Transport anti-amplification

IETF RFC 9000, QUIC: A UDP-Based Multiplexed and Secure Transport, 2021, especially sections 8 and 21. Establishes address-validation and amplification defenses. Application authorization, replay, and admission remain separate responsibilities. https://www.rfc-editor.org/rfc/rfc9000.html

Review conclusion: the revised specification removes unsupported guarantees and states enforceable targets. It remains a proposed architecture requiring implementation, model validation, adversarial testing, independent review, and sustained operator stewardship.